

Team Note of BunCua Team

MinhhHoangPersia - Pain - Lingg

Compiled on September 5, 2026

Contents

1	Template	1
2	Data Structure	1
2.1	Segment Tree Walk	1
2.2	Persistent Segment Tree	1
2.3	Trie	1
3	Graphs	2
3.1	2 SAT	2
3.2	LCA $O(1)$	2
3.3	Centroid Decomposition	3
3.4	Heavy Light Decomposition	3
4	DP	3
4.1	DP Digit Optimization 1	3
4.2	DP Digit Optimization 2	4
4.3	DP SoS	4
5	Strings	4
6	Math	4
7	Geometry	4
8	Miscellaneous	4

1 Template

2 Data Structure

2.1 Segment Tree Walk

```

/* Find first element in range [u, v] have value >= val */
int findPos(int id, int l, int r, int u, int v, ll val) {
    if(l > v || r < u) return n + 1;
    if(st[id] < val) return n + 1;
    if(l == r) return l;
    push(id);
    int mid = (l + r) / 2;
    int res = findPos(id * 2, l, mid, u, v, val);
    if(res == n + 1) {
        res = findPos(id * 2 + 1, mid + 1, r, u, v, val);
    }
    return res;
}

```

2.2 Persistent Segment Tree

```

struct node {
    node *l, *r;
    int v;

    node() {
        l = r = nullptr;
        v = 0;
    }
};

```

```

const int MAX_VAL = 300005;
node *t[MAX_VAL];

```

```

void refine(node *cur) {
    cur->v = cur->l->v + cur->r->v;
}

```

```

void build(node *cur, int l, int r) {
    cur->l = new node();

```

```

    cur->r = new node();
    if (l == r) {
        return;
    }
    int mid = (l + r) / 2;
    build(cur->l, l, mid);
    build(cur->r, mid + 1, r);
    refine(cur);
}

void update(node *cur, int l, int r, int u, int x) {
    if (l == r) {
        cur->v += x;
        return;
    }
    int mid = (l + r) / 2;
    if (u <= mid) {
        node *old = cur->l;
        cur->l = new node();
        cur->l->l = old->l;
        cur->l->r = old->r;
        cur->l->v = old->v;
        update(cur->l, l, mid, u, x);
    } else {
        node *old = cur->r;
        cur->r = new node();
        cur->r->l = old->l;
        cur->r->r = old->r;
        cur->r->v = old->v;
        update(cur->r, mid + 1, r, u, x);
    }
    refine(cur);
}

node* update_version(node *prev_root, int l, int r, int pos,
int val) {
    node *cur = new node();
    cur->l = prev_root->l;
    cur->r = prev_root->r;
    cur->v = prev_root->v;
    update(cur, l, r, pos, val);
    return cur;
}

int query(node *cur, int l, int r, int u, int v) {
    if (l > v || r < u) return 0;
    if (l >= u && r <= v) return cur->v;
    int mid = (l + r) / 2;
    return query(cur->l, l, mid, u, v) + query(cur->r, mid + 1,
r, u, v);
}

int query_diff(node *rootR, node *rootL, int l, int r, int u,
int v) {
    if (l > v || r < u) return 0;
    if (l >= u && r <= v) return rootR->v - rootL->v;
    int mid = (l + r) / 2;
    return query_diff(rootR->l, rootL->l, l, mid, u, v)
        + query_diff(rootR->r, rootL->r, mid + 1, r, u, v);
}

```

2.3 Trie

```

const int LOG_MAX = 29;

```

```

struct Node {
    Node* c[2];
    int cnt;

    Node() {

```

```

    c[0] = c[1] = nullptr;
    cnt = 0;
}
};

void insert(Node* root, int val) {
    Node* cur = root;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        if (cur->c[x] == nullptr) {
            cur->c[x] = new Node();
        }
        cur = cur->c[x];
        cur->cnt++;
    }
}

bool search(Node* root, int val) {
    Node* cur = root;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        if (cur->c[x] == nullptr || cur->c[x]->cnt == 0) {
            return false;
        }
        cur = cur->c[x];
    }
    return cur->cnt > 0;
}

int count_occurrences(Node* root, int val) {
    Node* cur = root;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        if (cur->c[x] == nullptr || cur->c[x]->cnt == 0) return 0;
        cur = cur->c[x];
    }
    return cur->cnt;
}

void erase(Node* root, int val) {
    if (!search(root, val)) return;
    Node* cur = root;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        cur = cur->c[x];
        cur->cnt--;
    }
}

void update(Node* root, int old_val, int new_val) {
    if (!search(root, old_val)) return;
    erase(root, old_val);
    insert(root, new_val);
}

```

3 Graphs

3.1 2 SAT

```

/*
- For a boolean variable x, we create two nodes: x (1..n)
  and NOT(x) (n+1..2n).
- A clause (x OR y) is equivalent to: (!x => y) and (!y => x).
- If x and NOT(x) are in the same Strongly Connected
  Component (SCC),
  it's a contradiction -> NO solution.
- We assign TRUE to x if its SCC is topologically sorted
  AFTER NOT(x)'s SCC.
  (Tarjan assigns SCC IDs in reverse topological order, so
  col[x] < col[NOT(x)])
*/

const int N = 2e4 + 5;

int n;
vector<int> G[2 * N];
int num[2 * N], low[2 * N], cnt;
int col[2 * N], ct;
vector<int> stk;
int ans[N];

```

```

int NOT(int x) {
    return (x <= n ? x + n : x - n);
}

void addedge(int x, int y) {
    G[NOT(x)].push_back(y);
    G[NOT(y)].push_back(x);
}

void tarjan(int u) {
    num[u] = low[u] = ++cnt;
    stk.push_back(u);
    for (int v : G[u]) {
        if (!col[v]) {
            if (num[v]) {
                low[u] = min(low[u], num[v]);
            } else {
                tarjan(v);
                low[u] = min(low[u], low[v]);
            }
        }
    }
    if (num[u] == low[u]) {
        ct++;
        while (true) {
            int v = stk.back();
            stk.pop_back();
            col[v] = ct;
            if (v == u) break;
        }
    }
}

bool solve() {
    rep(i, 1, 2 * n) {
        if (!num[i]) tarjan(i);
    }
    rep(i, 1, n) {
        if (col[i] == col[NOT(i)]) return false;
        ans[i] = (col[i] < col[NOT(i)]);
    }
    return true;
}

```

3.2 LCA O(1)

```

const int N = 1e5 + 5;
const int LOG = 18;

vector<int> G[N];
int tin[N], cnt, rn[N];
int euler[2 * N], ct, first[N];
int depth[N];
int rmq[LOG][2 * N];

void dfs(int u, int p = 0) {
    tin[u] = ++cnt;
    rn[cnt] = u;
    euler[++ct] = tin[u];
    first[tin[u]] = ct;
    for (int v : G[u]) {
        if (v == p) continue;
        depth[v] = depth[u] + 1;
        dfs(v, u);
        euler[++ct] = tin[u];
    }
}

void build_lca() {
    for (int i = 1; i <= ct; i++) {
        rmq[0][i] = euler[i];
    }
    for (int j = 1; j < LOG; j++) {
        for (int i = 1; i <= ct - (1 << (j - 1)); i++) {
            rmq[j][i] = min(rmq[j - 1][i], rmq[j - 1][i + (1 << (j - 1))]);
        }
    }
}

```

```
int lca(int x, int y) {
    x = first[tin[x]];
    y = first[tin[y]];
    if (x > y) swap(x, y);
    int k = 31 - __builtin_clz(y - x + 1);
    int min_tin = min(rmq[k][x], rmq[k][y - (1 << k) + 1]);
    return rn[min_tin];
}
```

3.3 Centroid Decomposition

```
const int N = 3e5 + 5;
vector<int> G[N];
int sz[N];
bool deleted[N];
int centroid_par[N];

void get_sz(int u, int par) {
    sz[u] = 1;
    for (int v : G[u]) {
        if (v != par && !deleted[v]) {
            get_sz(v, u);
            sz[u] += sz[v];
        }
    }
}

int get_centroid(int u, int par, int total_nodes) {
    for (int v : G[u]) {
        if (v != par && !deleted[v]) {
            if (sz[v] > total_nodes / 2) {
                return get_centroid(v, u, total_nodes);
            }
        }
    }
    return u;
}

void solve_centroid(int centroid) {
    // TODO: Add your logic here (e.g., DFS to collect all
    // distances from the
    // centroid to the nodes in its component, then count valid
    // pairs).
    //
    // Example structure:
    // for (int v : G[centroid]) {
    //     if (!deleted[v]) {
    //         // 1. DFS(v) to query answer with currently stored
    //         // paths
    //         // 2. DFS(v) again to add new paths to our data
    //         // structure
    //     }
    // }
    // // 3. Clear the data structure for the next centroid
}

void build_centroid(int u, int p = -1) {
    get_sz(u, -1);
    int centroid = get_centroid(u, -1, sz[u]);
    deleted[centroid] = true;
    centroid_par[centroid] = p;
    solve_centroid(centroid);
    for (int v : G[centroid]) {
        if (!deleted[v]) {
            build_centroid(v, centroid);
        }
    }
}
```

3.4 Heavy Light Decomposition

```
const int N = 2e5 + 5;
vector<int> G[N];
int sz[N], son[N], depth[N], pre[N];
int in[N], cnt, top[N];

void dfs1(int u, int par) {
    sz[u] = 1;
    son[u] = 0;
    pre[u] = par;
```

```
for (int v : G[u]) {
    if (v != par) {
        depth[v] = depth[u] + 1;
        dfs1(v, u);
        sz[u] += sz[v];
        if (sz[v] > sz[son[u]]) {
            son[u] = v;
        }
    }
}

void rebuild(int u, int topp) {
    in[u] = ++cnt;
    top[u] = topp;
    if (son[u]) {
        rebuild(son[u], topp);
    }
    for (int v : G[u]) {
        if (v != pre[u] && v != son[u]) {
            rebuild(v, v);
        }
    }
}

int query_path(int x, int y) {
    int res = 0;
    while (top[x] != top[y]) {
        if (depth[top[x]] < depth[top[y]]) {
            swap(x, y);
        }
        res = max(res, get(1, 1, n, in[top[x]], in[x]));
        x = pre[top[x]];
    }
    if (depth[x] > depth[y]) {
        swap(x, y);
    }
    res = max(res, get(1, 1, n, in[x], in[y]));
    return res;
}

void update_path(int x, int y, int val) {
    while (top[x] != top[y]) {
        if (depth[top[x]] < depth[top[y]]) {
            swap(x, y);
        }
        x = pre[top[x]];
    }
    if (depth[x] > depth[y]) {
        swap(x, y);
    }
}

/*
=====
USAGE EXAMPLE:
dfs1(1, 1);
cnt = 0;
rebuild(1, 1);
// for(int i = 1; i <= n; i++) update(1, 1, n, in[i],
// a[i]);
=====
*/
```

4 DP

4.1 DP Digit Optimization 1

```
/*
DIGIT DP OPTIMIZATION 1: REMOVING THE `is_less` (fr)
DIMENSION
*/

vector<int> R;
pair<ll, ll> dp[20];

pair<ll, ll> calc(int pos, int fr) {
    if (pos == -1) return make_pair(0, 1);
    auto &res = dp[pos];
    if (fr && res != make_pair(-1LL, -1LL)) return res;
    int lim_r = (fr ? 9 : R[pos]);
    pair<ll, ll> val = make_pair(0, 0);
```

```

for (int i = 0; i <= lim_r; i++) {
    pair<ll, ll> nxt = calc(pos - 1, fr | (i < lim_r));
    val.first += nxt.first + i * nxt.second;
    val.second += nxt.second;
}
return (fr ? res = val : val);
}

```

4.2 DP Digit Optimization 2

```

/*
    DIGIT DP OPTIMIZATION 2: ONE-TIME MEMSET FOR MULTI-TEST
    CASES
*/

const int mod = 1e9 + 7;

string R;
int dp[51][51][51];

int calc(int pos, int c3, int c6, int c9, int fr) {
    if (pos == -1) {
        return (c3 > 0 && c3 == c6 && c6 == c9);
    }
    auto &res = dp[pos][c3][c6][c9];
    if (fr && res != -1) return res;
    ll val = 0;
    int lim_r = (fr ? 9 : R[pos] - '0');
    for (int i = 0; i <= lim_r; i++) {
        val += calc(pos - 1, c3 + (i == 3), c6 + (i == 6), c9 + (i == 9), fr | (i < lim_r));
        if (val >= mod) val -= mod;
    }
    return (fr ? res = val : val);
}

int solve(string x) {
    reverse(x.begin(), x.end());
    R = x;
    return calc((int)R.size() - 1, 0, 0, 0, 0);
}

bool check(string x) {
    int c3 = 0, c6 = 0, c9 = 0;
    for (auto i : x) {
        c3 += (i == '3');
        c6 += (i == '6');
        c9 += (i == '9');
    }
    return (c3 > 0 && c3 == c6 && c6 == c9);
}

/*
// Usage in main():
// memset(dp, -1, sizeof dp); // Call this ONLY ONCE
// while (test--) {
//     string a, b; cin >> a >> b;
//     cout << (1LL * solve(b) - solve(a) + check(a) + mod) %
mod << "\n";
// }
*/

```

4.3 DP SoS

```

const int LOG = 20;
int dp[1 << LOG];

// 1. Sum over Submasks: dp[mask] = \sum_{sub \subseteq mask} A[sub]
// (sub is formed by turning OFF some bits of mask)
void sos_submask(int n, int a[]) {
    for (int mask = 0; mask < (1 << n); mask++) {
        dp[mask] = a[mask];
    }
    for (int i = 0; i < n; i++) {
        for (int mask = 0; mask < (1 << n); mask++) {
            if ((mask >> i) & 1) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
}

```

```

}
}

// 2. Sum over Supermasks: dp[mask] = \sum_{mask \subseteq sup} A[sup]
// (sup is formed by turning ON some bits of mask)
void sos_supermask(int n, int a[]) {
    for (int mask = 0; mask < (1 << n); mask++) {
        dp[mask] = a[mask];
    }
    for (int i = 0; i < n; i++) {
        for (int mask = 0; mask < (1 << n); mask++) {
            if (!(mask >> i) & 1) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
}

/*
COMMON USAGE PATTERNS & TRICKS:
1. Find pairs (u, v) such that u & v = 0:
2. Number of elements `x` such that (x & mask) == x:
3. Number of elements `x` such that (x & mask) == mask:
4. Finding two elements x, y with maximum (x | y):
*/

```

5 Strings

6 Math

7 Geometry

8 Miscellaneous